Encapsulation, Inheritance, Polymorphism, Abstraction

- Encapsulation
- Inheritance
  - diamond problem
- Polymorphism
  - Runtime polymorphism — method overriding
  - Compile-time polymorphism — method overloading
- Abstraction
- Exercice 1:
- Exercice 2:

Encapsulation

> "Encapsulation is hiding internal state behind private fields and exposing controlled access through public methods — it protects data integrity."

Without encapsulation

```
class BankAccount {
    double balance; // public!
}

BankAccount acc = new BankAccount();
acc.balance = -99999; // anyone can break it
```

With encapsulation

```
class BankAccount {
    private double balance;

    public void deposit(double amt) {
        if (amt > 0) balance += amt;
    }
    public double getBalance(){
        return balance;
    }
}
// acc.balance = -99999 → compile error!
```

Access modifiers — the encapsulation tools

```
public    // visible everywhere
protected // visible in same package + subclasses
(default) // visible in same package only
private   // visible in same class only  ← use this for fields
```

Inheritance

> A child class reuses and extends the behaviour of a parent class using `extends`.

```
class Animal {
    String name;
```

```
3
4       Animal(String name) { this.name = name; }
5
6       void breathe() {
7           System.out.println(name + " breathes");
8       }
9   }
10
11  class Dog extends Animal {       // inherits breathe()
12      Dog(String name) {
13          super(name);               // calls Animal constructor
14      }
15
16      void bark() {
17          System.out.println(name + " barks!");
18      }
19  }
20
21  Dog d = new Dog("Rex");
22  d.breathe(); // inherited from Animal
23  d.bark();    // Dog's own method
```

- Java supports **single inheritance** only — one class can extend only one parent
- `super` calls the parent constructor or method
- `final class` cannot be extended — String is final!
- Multiple inheritance via classes is forbidden — use interfaces instead

diamond problem

Pourquoi ça s'appelle "diamond" ?La forme du graphe ressemble à un diamant.

```
1         A
2        / \
3       B   C
4        \ /
5         D
```

Le diamond problem c'est quand une classe hérite du même méthode par deux chemins différents et Java ne sait pas lequel prendre. Pour les classes c'est interdit. Pour les interfaces tu dois override et choisir explicitement.

Polymorphism

> Same method call — different behaviour depending on the actual object type. Two flavours: compile-time and runtime.

Runtime polymorphism — method overriding

```
1   class Animal {
2       void speak() { System.out.println("..."); }
3   }
4   class Dog extends Animal {
5       @Override
6       void speak() { System.out.println("Woof!"); }
7   }
8   class Cat extends Animal {
```

```
    @Override
    void speak() { System.out.println("Meow!"); }
}

// Polymorphism in action — same type, different objects
Animal a1 = new Dog();
Animal a2 = new Cat();
a1.speak(); // "Woof!" — decided at RUNTIME
a2.speak(); // "Meow!" — decided at RUNTIME

// The real power — same loop, different behaviour
List<Animal> animals = List.of(new Dog(), new Cat(), new Dog());
animals.forEach(Animal::speak); // Woof! Meow! Woof!
```

Compile-time polymorphism — method overloading

```
class Calculator {
    int    add(int a,    int b)    { return a + b; }
    double add(double a, double b) { return a + b; }
    int    add(int a,    int b,    int c) { return a+b+c; }
}
// Same name, different signature — decided at COMPILE time
```

Abstraction

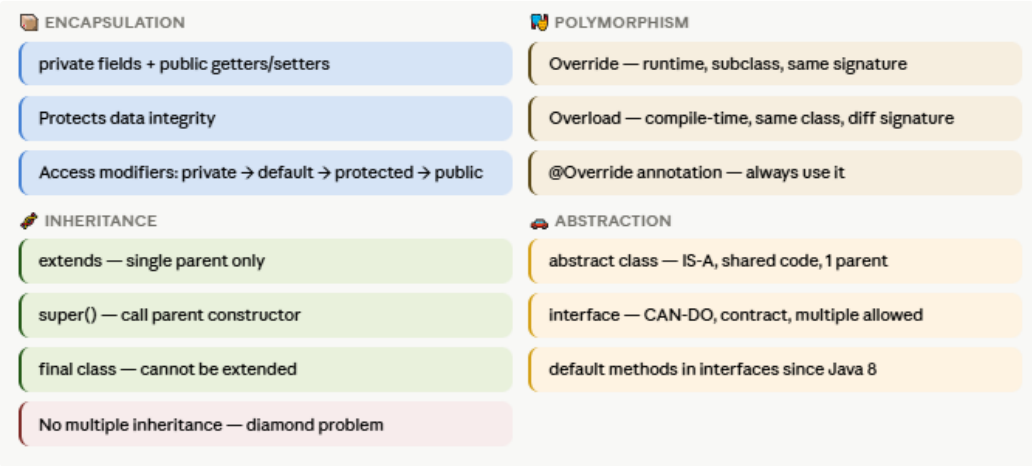
> Hide *how* something works. Show only *what* it does. Two tools: abstract class and interface.

```
abstract class Shape {
    String color;

    // concrete method — shared
    void setColor(String c){
        this.color = c;
    }

    // abstract — MUST override
    abstract double area();
}

class Circle extends Shape {
    double r;
    Circle(double r){ this.r=r; }

    @Override
    double area(){
        return Math.PI * r * r;
    }
}

interface Drawable {
    void draw();        // abstract

    // default method (Java 8+)
    default void print(){
        System.out.println("printing");
    }
}

class Circle extends Shape
             implements Drawable {
    @Override
    public void draw(){
```

```
        System.out.println("○");
    }
}
```

```
                 Abstract class      Interface
Fields           any type            public static final only
Constructor      yes                 no
Methods          abstract + concrete abstract + default (J8+)
extends/impl     extends (1 only)    implements (multiple!)
When to use      shared base code    contract / capability
```

Rule of thumb: "Is it a *type of*?" → abstract class. "Can it *do*?" → interface. Dog IS-A Animal. Dog CAN-DO Runnable.



Exercice 1:

```java
interface A {
    default void hello() {
        System.out.println("A");
    }
}
interface B {
    default void hello() {
        System.out.println("B");
    }
}
class C implements A, B {
    @Override
    public void hello() {
        A.super.hello();
    }
}
new C().hello();
```

When two interfaces have the same default method, the implementing class MUST override it — otherwise compile error. Here C overrides hello() and explicitly calls A.super.hello() → prints 'A'. This is how Java solves the interface diamond problem.

Exercice 2:

What is the problem with this code?

```java
class Person {
    private List<String> hobbies;

    Person(List<String> hobbies) {
        this.hobbies = hobbies;
    }
    public List<String> getHobbies() {
        return hobbies;
    }
}
```

private field is necessary but not sufficient! The getter returns the actual reference — caller can call getHobbies().add('hacking') and mutate internal state. Fix: return List.copyOf(hobbies) in the getter. This is the same defensive copy trap as in immutability.